

State Management Exercise

In a real life application your data would probably come from an API. You would probably have an empty array in the state and then use a method inside an action to fill it with data. Actions are meant to mutate (update value) your store's data.

- Start this exercise with the solution 04b_fetchAPIExercise
- For this application we will have a productStore.js and a cartStore.js. Together they will make up our Global Store.
- Start by installing Pinia in the terminal: `npm install pinia`
- Add the following in main.js

```
main.js > ...
import './assets/main.css'

import { createApp } from 'vue'
import { createPinia } from 'pinia'
import App from './App.vue'

const app = createApp(App)

app.use(createPinia())

app.mount('#app')
```

- Add the stores folder under src folder and create new the files: productStore.js and cartStore.js.
- In `productStore.js` make sure to have an empty products array in the state.
- Then create a `fill()` method in the actions section to populate the products array with data from <http://localhost:3000/products> using `fetch`.
- Make sure to use `async` and `await`. When you use `fetch` with `async/await`, you can initiate an HTTP request and wait for the response without blocking the main thread. This is particularly useful for fetching data from APIs or performing other network operations. By marking a function as `async`, you enable the use of the `await` keyword inside that function, which can be used to pause the execution until the `fetch` operation is complete.
- In `productsDisplay.vue` make sure to have the following and then user the `productStore` products array to access values:

```
import { useProductStore } from "@/stores/productStore";
const productStore = useProductStore();
productStore.fill();
```

- In `cartStore.js` make sure to have an empty cart array and the Boolean `primeMember` set to true.
- Then create the `updateCart(id)` method to add the products ids in the cart and `removeFromCart()` method to reset the length to 0. Both should be methods in the actions section.

Note about getters in Pinia.

Getters are equivalent to computed properties in components but in a store computed properties are used with complex logic that includes reactive data. The difference between a computed property and a method is that computed properties are cached meaning that it will re-evaluate only when some of its reactive dependencies have changed and not all the time like methods. In comparison, a method invocation will always run the function whenever a re-render happens.

Why do we need caching? Imagine we have an expensive computed property list, which requires looping through a huge array and doing a lot of computations. Then we may have other computed properties that in turn depend on list. Without caching, we would be executing list's getter many more times than necessary! In cases where you do not want caching, use a method call instead.

Getters are useful when you need to compute something from the state, for example, if you have a list of todos and you want to know how many are completed, you can create a getter for that.

If there is no computation just access the state directly

Getters . Pinia. (n.d.). <https://pinia.vuejs.org/core-concepts/getters.html>